I used ChatGPT and Gemini as my two GenAIs, I used the same copy-pasted prompt for both AIs as follows. I used them through their respective websites directly.

I pasted the code given to me, then used the following prompt
note that any formatting mistakes (such as asterisks italicizing things) are just pasting errors
this is the requirements, the first few are the requirements of the code and then come your job
• Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.
• Each solutionN.txt file should include the following:
o If no solutions are found, a single line: “No solution found.”
o If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.
o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.
• You must modify the Assignment 4 you receive, not your Assignment 4.
• Do not replace the entire file with your code. Your GTA will use the Github Blame button to check your modifications.
• Add your name to the Profile Comments as “Revised by:” along with a revision date.
• You must get all required functionality from Assignment 4 working, even if the student you inherited it from did not.

ChatGPT gave me the following program:

```cpp
#include <array>      // Provides std::array for fixed-size grids and bitmask tables.
#include <bitset>     // Provides std::bitset to count how many candidates a cell has.
#include <cctype>     // Provides std::isspace for ignoring whitespace in puzzle files.
#include <cstddef>    // Provides std::size_t for solution counts.
#include <cstdint>    // Provides std::uint16_t for compact bitmasks.
#include <fstream>    // Provides std::ifstream and std::ofstream for puzzle files.
#include <functional> // Provides std::function so the search can report solutions without printing.
#include <iostream>   // Provides std::cout and std::cerr for displaying output and errors.
#include <string>     // Provides std::string for filenames and messages.
#include <utility>    // Provides std::swap for reordering the list of empty cells.
#include <vector>     // Provides std::vector for the list of filenames.

// Define a class that keeps the Sudoku grid and solving behavior together.
class SudokuSolver {
private:
    static constexpr int SIZE = 9;       // Number of rows and columns in a Sudoku grid.
    static constexpr int BOX_SIZE = 3;   // Width and height of each 3-by-3 box.
    static constexpr int CELLS = SIZE * SIZE; // Total number of cells (81).

    using Mask = std::uint16_t; // A bitmask where bit d is set when digit d is used (bits 1-9).
    static constexpr Mask ALL_DIGITS = 0x3FE; // Bits 1 through 9 set: every digit is a candidate.
    using Grid = std::array<int, CELLS>; // A flat, row-major grid; 0 means blank, 1-9 are digits.
    using SolutionHandler = std::function<void(const Grid&, std::size_t)>; // Called once for each solution found.

    Grid grid{}; // The current puzzle, filled in and undone as the search runs.
    std::array<Mask, SIZE> rowUsed{}; // Digits already used in each row.
    std::array<Mask, SIZE> colUsed{}; // Digits already used in each column.
    std::array<Mask, SIZE> boxUsed{}; // Digits already used in each 3-by-3 box.
    std::array<int, CELLS> empties{}; // Indices of the blank cells; the search reorders this list.
    int emptyCount = 0; // How many blank cells the puzzle has.
    std::size_t solutionCount = 0; // How many solutions the search has found so far.

    int boxOf(int row, int col) const { // Convert a row and column into a box number 0-8.
        return (row / BOX_SIZE) * BOX_SIZE + (col / BOX_SIZE); // Boxes are numbered left to right, top to bottom.
    } // End of boxOf.

    Mask candidatesFor(int cell) const { // Find every digit that can legally go in one cell.
        int row = cell / SIZE; // Row of this cell.
        int col = cell % SIZE; // Column of this cell.
        Mask used = rowUsed[row] | colUsed[col] | boxUsed[boxOf(row, col)]; // Combine digits used in the row, column, and box.
        return static_cast<Mask>(ALL_DIGITS & ~used); // Candidates are the digits that are not used anywhere.
    } // End of candidatesFor.

    bool prepareSearch(std::string& conflictMessage) { // Build the bitmasks and check the given clues.
        rowUsed.fill(0); // Forget any digits recorded for a previous puzzle.
        colUsed.fill(0); // Forget any digits recorded for a previous puzzle.
        boxUsed.fill(0); // Forget any digits recorded for a previous puzzle.
        emptyCount = 0; // Restart the list of blank cells.

        for (int cell = 0; cell < CELLS; ++cell) { // Visit every cell in reading order.
            int value = grid[cell]; // Read the digit (or 0) stored in this cell.

            if (value == 0) { // Check whether this cell is blank.
                empties[emptyCount++] = cell; // Remember the blank cell so the search can fill it later.
                continue; // Blank cells cannot conflict with anything.
            } // Finish handling a blank cell.

            int row = cell / SIZE; // Row of this given digit.
            int col = cell % SIZE; // Column of this given digit.
            int box = boxOf(row, col); // Box of this given digit.
            Mask bit = static_cast<Mask>(1u << value); // Bit that represents this digit.

            if ((rowUsed[row] | colUsed[col] | boxUsed[box]) & bit) { // Check whether the digit repeats in its row, column, or box.
                conflictMessage = "digit " + std::to_string(value) + " at row " + std::to_string(row + 1)
                                + ", column " + std::to_string(col + 1) + " repeats in its row, column, or box";
                return false; // Report that the given clues contradict each other.
            } // Finish handling a conflicting digit.

            rowUsed[row] |= bit; // Record the digit as used in its row.
```

```
            colUsed[col] |= bit; // Record the digit as used in its column.
            boxUsed[box] |= bit; // Record the digit as used in its box.
        } // Finish checking every cell.

        return true; // Confirm that all given digits are consistent.
    } // End of prepareSearch.

    void search(int depth, const SolutionHandler& onSolution) { // Recursively fill blank cells using backtracking.
        if (depth == emptyCount) { // Check whether every blank cell has been filled.
            ++solutionCount; // Count this complete solution.
            onSolution(grid, solutionCount); // Hand the solution to the caller instead of printing here.
            return; // End this recursive path because the grid is solved.
        } // Finish handling a completed grid.

        int bestIndex = depth; // Position in the list of the most-constrained blank cell found so far.
        int bestCount = SIZE + 1; // Start above the maximum so any real cell beats it.
        Mask bestMask = 0; // Candidates of the most-constrained cell found so far.

        for (int i = depth; i < emptyCount; ++i) { // Examine every blank cell that is still unfilled.
            Mask mask = candidatesFor(empties[i]); // Find the legal digits for this blank cell.
            int count = static_cast<int>(std::bitset<16>(mask).count()); // Count how many legal digits it has.

            if (count == 0) { // Check whether this cell has no legal digit at all.
                return; // Abandon this branch immediately because it cannot lead to a solution.
            } // Finish handling a dead end.

            if (count < bestCount) { // Check whether this cell is more constrained than the best so far.
                bestCount = count; // Remember how constrained it is.
                bestIndex = i; // Remember where it is in the list.
                bestMask = mask; // Remember its candidates.

                if (count == 1) { // Check whether the cell has exactly one choice.
                    break; // Stop looking because a forced cell cannot be beaten.
                } // Finish handling a forced cell.
            } // Finish comparing against the best cell.
        } // Finish looking for the most-constrained cell.

        std::swap(empties[depth], empties[bestIndex]); // Move the chosen cell to the current depth in the list.
        int cell = empties[depth]; // Read the cell this level will fill.
        int row = cell / SIZE; // Row of the chosen cell.
        int col = cell % SIZE; // Column of the chosen cell.
        int box = boxOf(row, col); // Box of the chosen cell.

        for (int digit = 1; digit <= SIZE; ++digit) { // Try each possible digit from 1 through 9.
            Mask bit = static_cast<Mask>(1u << digit); // Bit that represents this digit.

            if (!(bestMask & bit)) { // Check whether this digit is not a legal candidate.
                continue; // Skip it without re-checking the row, column, and box.
            } // Finish skipping an illegal digit.

            grid[cell] = digit; // Tentatively place this candidate in the cell.
            rowUsed[row] |= bit; // Mark the digit as used in the row.
            colUsed[col] |= bit; // Mark the digit as used in the column.
            boxUsed[box] |= bit; // Mark the digit as used in the box.

            search(depth + 1, onSolution); // Recursively try to complete the grid with this candidate.

            grid[cell] = 0; // Undo the tentative choice in the grid.
            rowUsed[row] ^= bit; // Free the digit in the row.
            colUsed[col] ^= bit; // Free the digit in the column.
            boxUsed[box] ^= bit; // Free the digit in the box.
        } // Finish trying all candidates for this cell.
    } // End of search.

```

```
    void printGrid(const Grid& board) const { // Print a grid in a readable Sudoku layout to the console.
        for (int row = 0; row < SIZE; ++row) { // Print the grid one row at a time.
            if (row > 0 && row % BOX_SIZE == 0) { // Check whether a horizontal box divider belongs before this row.
                std::cout << "------+-------+------\n"; // Separate the three horizontal bands of boxes.
            } // Finish checking for a horizontal divider.

            for (int col = 0; col < SIZE; ++col) { // Print every cell in the current row.
                if (col > 0 && col % BOX_SIZE == 0) { // Check whether a vertical box divider belongs here.
                    std::cout << "| "; // Separate the three vertical boxes.
                } // Finish checking for a vertical divider.

                int value = board[row * SIZE + col]; // Read the cell from the flat grid.
                if (value == 0) { // Check whether the cell is blank.
                    std::cout << "_ "; // Show an underscore for an empty cell.
                } else { // Handle a cell containing a digit.
                    std::cout << value << ' '; // Print the digit followed by a space.
                } // Finish printing this cell.
            } // Finish printing all cells in this row.

            std::cout << '\n'; // Move to the next output line after this row.
        } // Finish printing all nine rows.
    } // End of printGrid.

    // Write a completed grid in the same 9-characters-per-row format as the puzzle files.
    void writeGrid(std::ostream& output, const Grid& board) const {
        for (int row = 0; row < SIZE; ++row) {
            for (int col = 0; col < SIZE; ++col) {
                int value = board[row * SIZE + col];
                output << (value == 0 ? '_' : static_cast<char>('0' + value));
            }
            output << '\n';
        }
    }

public:
    bool loadFromFile(const std::string& filename) { // Load a puzzle file and check its format.
        std::ifstream inputFile(filename); // Attempt to open the requested puzzle file.
        if (!inputFile) { // Check whether the file could not be opened.
            std::cerr << "Error: could not open " << filename << ".\n"; // Explain which file failed to open.
            return false; // Tell the caller that loading failed.
        } // Finish handling an open failure.

        Grid loaded{}; // Read into a temporary grid so a bad file never changes the current puzzle.
        int cellCount = 0; // Count how many cell symbols have been read so far.
        char character; // Hold each character read from the input file.

        while (inputFile.get(character)) { // Read the file one character at a time.
            if (std::isspace(static_cast<unsigned char>(character))) { // Ignore spaces, tabs, and newline characters.
                continue; // Move to the next character without treating whitespace as a cell.
            } // Finish skipping whitespace.

            if (cellCount == CELLS) { // Check whether the file has a symbol beyond the 81st cell.
                std::cerr << "Error: " << filename << " contains more than 81 cells.\n"; // Explain the excessive cell count.
                return false; // Reject a puzzle with too many cells.
            } // Finish checking the maximum number of cells.

            int value = 0; // Hold the numeric value of this cell (0 for blank).
            if (character == '_') { // Check whether the character marks a blank cell.
                value = 0; // Store zero internally to represent the blank.
            } else if (character >= '1' && character <= '9') { // Check whether the character is a valid Sudoku digit.
                value = character - '0'; // Convert the character digit into its integer value.
            } else { // Handle any character that is not whitespace, underscore, or digit 1-9.
                std::cerr << "Error: invalid character '" << character << "' in " << filename
                          << " (row " << cellCount / SIZE + 1 << ", column " << cellCount % SIZE + 1 << ").\n";
                return false; // Stop loading because the puzzle format is invalid.
            } // Finish interpreting this input character.

            loaded[cellCount++] = value; // Store the cell and advance to the next position.
        } // Finish reading the complete file.
```

```
205
206         if (cellCount != CELLS) { // Check whether exactly 81 cells were read.
207             std::cerr << "Error: " << filename << " must contain exactly 81 cells but has " << cellCount << ".\n";
208             return false; // Reject a puzzle with too few cells.
209         } // Finish checking the cell count.
210
211         grid = loaded; // Accept the puzzle by copying the temporary grid into place.
212         return true; // Report that the puzzle file was loaded successfully.
213     } // End of loadFromFile.
214
215     // Solve the loaded puzzle, retain the original console output, and save all solutions to a file.
216     bool solveAndPrint(const std::string& filename, const std::string& solutionFilename) {
217         solutionCount = 0; // Reset the count left over from any previous puzzle.
218
219         std::cout << "\nPuzzle file: " << filename << "\n"; // Display the name of the current puzzle file.
220         std::cout << "Original puzzle:\n"; // Label the puzzle as it appeared in the file.
221         printGrid(grid); // Display the starting puzzle before attempting to solve it.
222
223         std::ofstream solutionFile(solutionFilename); // Create or overwrite the corresponding solution file.
224         if (!solutionFile) {
225             std::cerr << "Error: could not create " << solutionFilename << ".\n";
226             return false;
227         }
228
229         std::string conflictMessage; // Receive a description if the given clues contradict each other.
230         if (!prepareSearch(conflictMessage)) { // Build the bitmasks and reject conflicting clues.
231             std::cout << "No solution found (the given clues conflict: " << conflictMessage << ").\n";
232             solutionFile << "No solution found.\n"; // The required output for a puzzle with conflicting clues.
233             return static_cast<bool>(solutionFile);
234         } // Finish validating the starting puzzle.
235
236         auto printSolution = [this, &solutionFile](const Grid& solved, std::size_t number) {
237             std::cout << "\nSolution " << number << ":\n"; // Label the solution with a human-friendly number.
238             printGrid(solved); // Print the completed Sudoku grid.
239
240             if (number > 1) {
241                 solutionFile << '\n'; // Separate multiple solution grids with a blank line.
242             }
243             writeGrid(solutionFile, solved); // Save this complete solution in the input-file grid format.
244         }; // Finish defining the solution printer.
245
246         search(0, printSolution); // Run the depth-first search; solutions print and save as they are found.
247
248         if (solutionCount == 0) { // Check whether the search failed to find any complete solution.
249             std::cout << "No solution found.\n"; // Print the required message when no solution exists.
250             solutionFile << "No solution found.\n"; // Write the required no-solution message to the output file.
251         } else { // Handle one or more solutions.
252             std::cout << "\nNumber of solutions found: " << solutionCount << "\n";
253             solutionFile << "Solutions found: " << solutionCount << '\n'; // Required final line in the output file.
254         } // Finish handling the search result.
255
256         return static_cast<bool>(solutionFile); // Report whether writing the output file succeeded.
257     } // End of solveAndPrint.
258 }; // End of the SudokuSolver class.
259
260 // Create the output filename that corresponds to an input puzzle filename.
261 std::string makeSolutionFilename(const std::string& puzzleFilename, std::size_t puzzleNumber) {
262     std::size_t slashPosition = puzzleFilename.find_last_of("/\\");
263     std::size_t basePosition = (slashPosition == std::string::npos) ? 0 : slashPosition + 1;
264     std::string outputFilename = puzzleFilename;
265
266     // Replace a leading "puzzle" in the filename with "solution", preserving its number and extension.
267     if (outputFilename.compare(basePosition, 6, "puzzle") == 0) {
268         outputFilename.replace(basePosition, 6, "solution");
```

```
        return outputFilename;
    }

    // For a differently named input file, use its position in the argument list.
    return "solution" + std::to_string(puzzleNumber) + ".txt";
}

// Define the program's entry point, where execution begins.
int main(int argc, char* argv[]) { // Accept optional puzzle filenames from the command line.
    std::vector<std::string> filenames; // Hold the puzzle files to process.

    for (int i = 1; i < argc; ++i) { // Visit each command-line argument.
        filenames.push_back(argv[i]); // Treat each argument as a puzzle filename.
    } // Finish collecting command-line filenames.

    if (filenames.empty()) { // Check whether the user gave no filenames.
        filenames = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
    } // Finish choosing the default filenames.

    SudokuSolver solver; // Create one solver object to process each puzzle in sequence.
    bool allSuccessful = true; // Track whether every file loaded and its solution output was written.

    for (std::size_t i = 0; i < filenames.size(); ++i) { // Process each puzzle file.
        const std::string& filename = filenames[i];
        std::string solutionFilename = makeSolutionFilename(filename, i + 1);

        if (solver.loadFromFile(filename)) { // Continue only if the current file was read successfully.
            if (!solver.solveAndPrint(filename, solutionFilename)) {
                allSuccessful = false; // Remember that an output file could not be written.
            }
        } else { // Handle a file that could not be loaded or had an invalid format.
            std::cout << "Skipping " << filename << " because it could not be loaded.\n";
            allSuccessful = false; // Remember that at least one file failed.
        } // Finish handling the current file's loading result.
    } // Finish processing all puzzle files.

    return allSuccessful ? 0 : 1; // Return a nonzero exit status if any file failed.
}
```

Gemini produced the following program:

```cpp
// Include the tools this program needs, each with the reason it is needed.
#include <array> // Provides std::array for fixed-size grids and bitmask tables.
#include <bitset> // Provides std::bitset to count how many candidates a cell has.
#include <cctype> // Provides std::isspace for ignoring whitespace in puzzle files.
#include <cstddef> // Provides std::size_t for solution counts.
#include <cstdint> // Provides std::uint16_t for compact bitmasks.
#include <fstream> // Provides std::ifstream and std::ofstream for opening and writing files.
#include <functional> // Provides std::function so the search can report solutions without printing.
#include <iostream> // Provides std::cout and std::cerr for displaying output and errors.
#include <string> // Provides std::string for filenames and messages.
#include <utility> // Provides std::swap for reordering the list of empty cells.
#include <vector> // Provides std::vector for the list of filenames.

// Define a class that keeps the Sudoku grid and solving behavior together.
class SudokuSolver { // Start of the object-oriented Sudoku solver class.
private: // Keep the grid and helper methods internal to this class.
    static constexpr int SIZE = 9; // Number of rows and columns in a Sudoku grid.
    static constexpr int BOX_SIZE = 3; // Width and height of each 3-by-3 box.
    static constexpr int CELLS = SIZE * SIZE; // Total number of cells (81).
    using Mask = std::uint16_t; // A bitmask where bit d is set when digit d is used (bits 1-9).
    static constexpr Mask ALL_DIGITS = 0x3FE; // Bits 1 through 9 set: every digit is a candidate.
    using Grid = std::array<int, CELLS>; // A flat, row-major grid; 0 means blank, 1-9 are digits.
    using SolutionHandler = std::function<void(const Grid&, std::size_t)>; // Called once for each solution found.

    Grid grid{}; // The current puzzle, filled in and undone as the search runs.
    std::array<Mask, SIZE> rowUsed{}; // Digits already used in each row.
    std::array<Mask, SIZE> colUsed{}; // Digits already used in each column.
    std::array<Mask, SIZE> boxUsed{}; // Digits already used in each 3-by-3 box.
    std::array<int, CELLS> empties{}; // Indices of the blank cells; the search reorders this list.
    int emptyCount = 0; // How many blank cells the puzzle has.
    std::size_t solutionCount = 0; // How many solutions the search has found so far.

    int boxOf(int row, int col) const { // Convert a row and column into a box number 0-8.
        return (row / BOX_SIZE) * BOX_SIZE + (col / BOX_SIZE); // Boxes are numbered left to right, top to bottom.
    } // End of boxOf.

    Mask candidatesFor(int cell) const { // Find every digit that can legally go in one cell.
        int row = cell / SIZE; // Row of this cell.
        int col = cell % SIZE; // Column of this cell.
        Mask used = rowUsed[row] | colUsed[col] | boxUsed[boxOf(row, col)]; // Combine digits used in the row, column, and box.
        return static_cast<Mask>(ALL_DIGITS & ~used); // Candidates are the digits that are not used anywhere.
    } // End of candidatesFor.

    bool prepareSearch(std::string& conflictMessage) { // Build the bitmasks and check the given clues.
        rowUsed.fill(0); // Forget any digits recorded for a previous puzzle.
        colUsed.fill(0); // Forget any digits recorded for a previous puzzle.
        boxUsed.fill(0); // Forget any digits recorded for a previous puzzle.
        emptyCount = 0; // Restart the list of blank cells.
        for (int cell = 0; cell < CELLS; ++cell) { // Visit every cell in reading order.
            int value = grid[cell]; // Read the digit (or 0) stored in this cell.
            if (value == 0) { // Check whether this cell is blank.
                empties[emptyCount++] = cell; // Remember the blank cell so the search can fill it later.
                continue; // Blank cells cannot conflict with anything.
            } // Finish handling a blank cell.
            int row = cell / SIZE; // Row of this given digit.
            int col = cell % SIZE; // Column of this given digit.
            int box = boxOf(row, col); // Box of this given digit.
            Mask bit = static_cast<Mask>(1u << value); // Bit that represents this digit.
            if ((rowUsed[row] | colUsed[col] | boxUsed[box]) & bit) { // Check whether the digit repeats in its row, column, or box.
                conflictMessage = "digit " + std::to_string(value) + " at row " + std::to_string(row + 1) // Start describing the conflict.
                                + ", column " + std::to_string(col + 1) + " repeats in its row, column, or box"; // Finish describing it.
                return false; // Report that the given clues contradict each other.
            } // Finish handling a conflicting digit.
            rowUsed[row] |= bit; // Record the digit as used in its row.
            colUsed[col] |= bit; // Record the digit as used in its column.
            boxUsed[box] |= bit; // Record the digit as used in its box.
        } // Finish checking every cell.
        return true; // Confirm that all given digits are consistent.
    } // End of prepareSearch.
```

```
    void search(int depth, const SolutionHandler& onSolution) { // Recursively fill blank cells using backtracking.
        if (depth == emptyCount) { // Check whether every blank cell has been filled.
            ++solutionCount; // Count this complete solution.
            onSolution(grid, solutionCount); // Hand the solution to the caller instead of printing here.
            return; // End this recursive path because the grid is solved.
        } // Finish handling a completed grid.
        int bestIndex = depth; // Position in the list of the most-constrained blank cell found so far.
        int bestCount = SIZE + 1; // Start above the maximum so any real cell beats it.
        Mask bestMask = 0; // Candidates of the most-constrained cell found so far.
        for (int i = depth; i < emptyCount; ++i) { // Examine every blank cell that is still unfilled.
            Mask mask = candidatesFor(empties[i]); // Find the legal digits for this blank cell.
            int count = static_cast<int>(std::bitset<16>(mask).count()); // Count how many legal digits it has.
            if (count == 0) { // Check whether this cell has no legal digit at all.
                return; // Abandon this branch immediately because it cannot lead to a solution.
            } // Finish handling a dead end.
            if (count < bestCount) { // Check whether this cell is more constrained than the best so far.
                bestCount = count; // Remember how constrained it is.
                bestIndex = i; // Remember where it is in the list.
                bestMask = mask; // Remember its candidates.
                if (count == 1) { // Check whether the cell has exactly one choice.
                    break; // Stop looking because a forced cell cannot be beaten.
                } // Finish handling a forced cell.
            } // Finish comparing against the best cell.
        } // Finish looking for the most-constrained cell.
        std::swap(empties[depth], empties[bestIndex]); // Move the chosen cell to the current depth in the list.
        int cell = empties[depth]; // Read the cell this level will fill.
        int row = cell / SIZE; // Row of the chosen cell.
        int col = cell % SIZE; // Column of the chosen cell.
        int box = boxOf(row, col); // Box of the chosen cell.
        for (int digit = 1; digit <= SIZE; ++digit) { // Try each possible digit from 1 through 9.
            Mask bit = static_cast<Mask>(1u << digit); // Bit that represents this digit.
            if (!(bestMask & bit)) { // Check whether this digit is not a legal candidate.
                continue; // Skip it without re-checking the row, column, and box.
            } // Finish skipping an illegal digit.
            grid[cell] = digit; // Tentatively place this candidate in the cell.
            rowUsed[row] |= bit; // Mark the digit as used in the row.
            colUsed[col] |= bit; // Mark the digit as used in the column.
            boxUsed[box] |= bit; // Mark the digit as used in the box.
            search(depth + 1, onSolution); // Recursively try to complete the grid with this candidate.
            grid[cell] = 0; // Undo the tentative choice in the grid.
            rowUsed[row] ^= bit; // Free the digit in the row.
            colUsed[col] ^= bit; // Free the digit in the column.
            boxUsed[box] ^= bit; // Free the digit in the box.
        } // Finish trying all candidates for this cell.
    } // End of search.

    void printGrid(const Grid& board) const { // Print a grid in a readable Sudoku layout.
        for (int row = 0; row < SIZE; ++row) { // Print the grid one row at a time.
            if (row > 0 && row % BOX_SIZE == 0) { // Check whether a horizontal box divider belongs before this row.
                std::cout << "------+-------+------\n"; // Separate the three horizontal bands of boxes.
            } // Finish checking for a horizontal divider.
            for (int col = 0; col < SIZE; ++col) { // Print every cell in the current row.
                if (col > 0 && col % BOX_SIZE == 0) { // Check whether a vertical box divider belongs here.
                    std::cout << "| "; // Separate the three vertical boxes.
                } // Finish checking for a vertical divider.
                int value = board[row * SIZE + col]; // Read the cell from the flat grid.
                if (value == 0) { // Check whether the cell is blank.
                    std::cout << "_ "; // Show an underscore for an empty cell.
                } else { // Handle a cell containing a digit.
                    std::cout << value << ' '; // Print the digit followed by a space.
                } // Finish printing this cell.
            } // Finish printing all cells in this row.
            std::cout << '\n'; // Move to the next output line after this row.
```

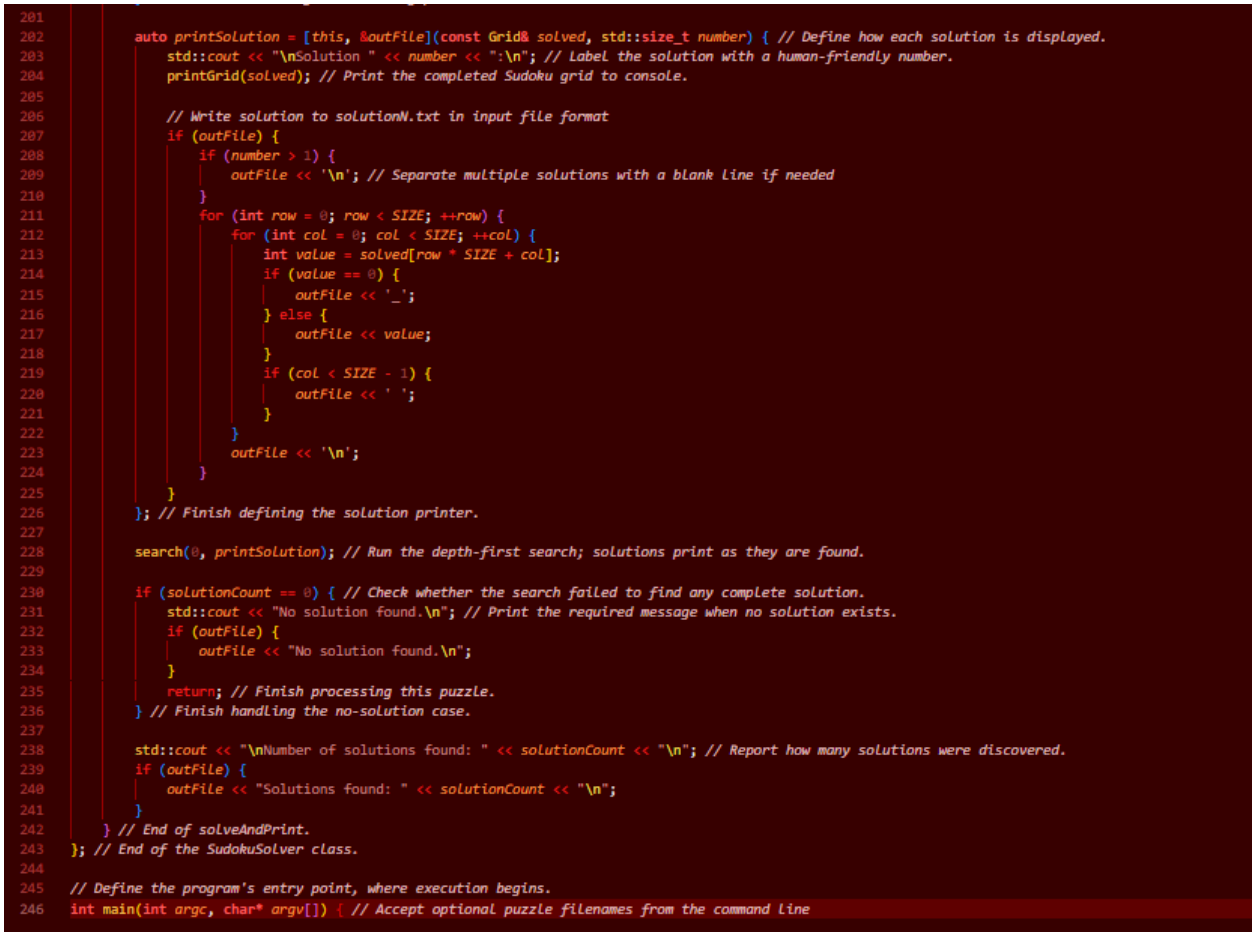
```
        } // Finish printing all nine rows.
    } // End of printGrid.

public: // Expose the operations that main needs to use.
    bool loadFromFile(const std::string& filename) { // Load a puzzle file and check its format.
        std::ifstream inputFile(filename); // Attempt to open the requested puzzle file.
        if (!inputFile) { // Check whether the file could not be opened.
            std::cerr << "Error: could not open " << filename << ".\n"; // Explain which file failed to open.
            return false; // Tell the caller that loading failed.
        } // Finish handling an open failure.
        Grid loaded{}; // Read into a temporary grid so a bad file never changes the current puzzle.
        int cellCount = 0; // Count how many cell symbols have been read so far.
        char character; // Hold each character read from the input file.
        while (inputFile.get(character)) { // Read the file one character at a time.
            if (std::isspace(static_cast<unsigned char>(character))) { // Ignore spaces, tabs, and newline characters.
                continue; // Move to the next character without treating whitespace as a cell.
            } // Finish skipping whitespace.
            if (cellCount == CELLS) { // Check whether the file has a symbol beyond the 81st cell.
                std::cerr << "Error: " << filename << " contains more than 81 cells.\n"; // Explain the excessive cell count.
                return false; // Reject a puzzle with too many cells.
            } // Finish checking the maximum number of cells.
            int value = 0; // Hold the numeric value of this cell (0 for blank).
            if (character == '_') { // Check whether the character marks a blank cell.
                value = 0; // Store zero internally to represent the blank.
            } else if (character >= '1' && character <= '9') { // Check whether the character is a valid Sudoku digit.
                value = character - '0'; // Convert the character digit into its integer value.
            } else { // Handle any character that is not whitespace, underscore, or digit 1-9.
                std::cerr << "Error: invalid character '" << character << "' in " << filename // Identify the bad character and file.
                          << " (row " << cellCount / SIZE + 1 << ", column " << cellCount % SIZE + 1 << ").\n"; // Say exactly where it is.
                return false; // Stop loading because the puzzle format is invalid.
            } // Finish interpreting this input character.
            loaded[cellCount++] = value; // Store the cell and advance to the next position.
        } // Finish reading the complete file.
        if (cellCount != CELLS) { // Check whether exactly 81 cells were read.
            std::cerr << "Error: " << filename << " must contain exactly 81 cells but has " << cellCount << ".\n"; // Explain the required size and the actual count.
            return false; // Reject a puzzle that has too few cells.
        } // Finish checking the cell count.
        grid = loaded; // Accept the puzzle by copying the temporary grid into place.
        return true; // Report that the puzzle file was loaded successfully.
    } // End of loadFromFile.

    void solveAndPrint(const std::string& filename) { // Solve the loaded puzzle and print its results.
        solutionCount = 0; // Reset the count left over from any previous puzzle.
        std::cout << "\nPuzzle file: " << filename << "\n"; // Display the name of the current puzzle file.
        std::cout << "Original puzzle:\n"; // Label the puzzle as it appeared in the file.
        printGrid(grid); // Display the starting puzzle before attempting to solve it.

        // Determine solution filename (e.g., solution1.txt from puzzle1.txt)
        std::string nStr = "";
        for (char c : filename) {
            if (std::isdigit(static_cast<unsigned char>(c))) {
                nStr += c;
            }
        }
        std::string solutionFilename = "solution" + (nStr.empty() ? "1" : nStr) + ".txt";
        std::ofstream outFile(solutionFilename);

        std::string conflictMessage; // Receive a description if the given clues contradict each other.
        if (!prepareSearch(conflictMessage)) { // Build the bitmasks and reject conflicting clues.
            std::cout << "No solution found (the given clues conflict: " << conflictMessage << ").\n"; // Report why there is no solution.
            if (outFile) {
                outFile << "No solution found.\n";
            }
            return; // Skip the search because conflicting clues cannot produce a valid solution.
        } // Finish validating the starting puzzle.
```



```
201
202         auto printSolution = [this, &outFile](const Grid& solved, std::size_t number) { // Define how each solution is displayed.
203             std::cout << "\nSolution " << number << ":\n"; // Label the solution with a human-friendly number.
204             printGrid(solved); // Print the completed Sudoku grid to console.
205
206             // Write solution to solutionN.txt in input file format
207             if (outFile) {
208                 if (number > 1) {
209                     outFile << '\n'; // Separate multiple solutions with a blank line if needed
210                 }
211                 for (int row = 0; row < SIZE; ++row) {
212                     for (int col = 0; col < SIZE; ++col) {
213                         int value = solved[row * SIZE + col];
214                         if (value == 0) {
215                             outFile << '_';
216                         } else {
217                             outFile << value;
218                         }
219                         if (col < SIZE - 1) {
220                             outFile << ' ';
221                         }
222                     }
223                     outFile << '\n';
224                 }
225             }
226         }; // Finish defining the solution printer.
227
228         search(0, printSolution); // Run the depth-first search; solutions print as they are found.
229
230         if (solutionCount == 0) { // Check whether the search failed to find any complete solution.
231             std::cout << "No solution found.\n"; // Print the required message when no solution exists.
232             if (outFile) {
233                 outFile << "No solution found.\n";
234             }
235             return; // Finish processing this puzzle.
236         } // Finish handling the no-solution case.
237
238         std::cout << "\nNumber of solutions found: " << solutionCount << "\n"; // Report how many solutions were discovered.
239         if (outFile) {
240             outFile << "Solutions found: " << solutionCount << "\n";
241         }
242     } // End of solveAndPrint.
243 }; // End of the SudokuSolver class.
244
245 // Define the program's entry point, where execution begins.
246 int main(int argc, char* argv[]) { // Accept optional puzzle filenames from the command line
```

2E:

- ChatGPT
    - Correctness: The program correctly solves the puzzle and validates the starting grid
    - Execution time: Both scripts solve in similar time. ChatGPT's program solves quicker because it chooses the empty cell with the fewest legal possibilities
    - Space complexity: Both scripts use very little memory.
    - Maintainability: ChatGPT's code has an organized structure and its functions have clearer responsibilities
- Gemini
    - Correctness: The program correctly solves the puzzle and validates the starting grid

- Execution time: Both scripts solve in similar time. Gemini does not use the system ChatGPT does for empty cells, however.
- Space complexity: Both scripts use very little memory.
- Maintainability: Gemini has similar structure but is less clear on its functions

2F: I would choose ChatGPT's script since it has easier responsibility separation and is slightly better on execution time

2G: I would verify the output formatting matches the input files and reduce the excessive comments.